# Dayli AI — Database Schema Documentation

**Version:** v4 | **Database:** Supabase (PostgreSQL + pgvector) | **Date:** April 2026

---

## Table of Contents


1. profiles
2. solutions
3. solution_embeddings
4. community_ratings
5. solution_likes
6. collections
7. collection_items
8. solution_pathways
9. assessment_sessions
10. needs_assessments
11. challenge_categories
12. solution_feedback
13. diagnostic_sessions
14. diagnostic_prompts
15. diagnostic_paths
16. diagnostic_results
17. solution_results
18. solution_tags
19. filter_options
20. SQL Functions


---

## 1. profiles

**Purpose:** Stores user profile information linked to Supabase Auth. A profile is auto-created when a user signs up via a database trigger.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, FK → auth.users(id), ON DELETE CASCADE | Links directly to the Supabase auth user ID |
| `display_name` | text | — | User's display name (defaults to email prefix on signup) |
| `avatar_url` | text | — | URL to the user's avatar image |
| `role` | text | DEFAULT 'user' | User role in the system (e.g. 'user', 'admin') |
| `created_at` | timestamptz | DEFAULT now() | When the profile was created |
| `updated_at` | timestamptz | DEFAULT now() | Auto-updated via trigger on every row change |

**RLS:** Users can only view, insert, and update their own profile.

**Triggers:**

- `on_auth_user_created` — auto-creates a profile row when a new auth.users row is inserted
- `on_profile_updated` — auto-updates `updated_at` on every change

---

## 2. solutions

**Purpose:** The core catalog of assistive solutions — products, techniques, and resources that help people with activities of daily living (ADLs). Originally named `internal_solutions`, renamed in migration 003.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier for each solution |

| | | | |
|---|---|---|---|
| `title` | text | — | Name of the solution (e.g. "Handheld Showerhead with Long Hose") |
| `description` | text | — | Detailed explanation of what the solution is and how it helps |
| `source_type` | text | — | Where the solution came from: 'web', 'community', 'youtube' |
| `adl_category` | text | — | The ADL area this solution addresses: bathing, dressing, eating, mobility, toileting, transferring, communication, memory |
| `disability_tags` | text[] | — | Array of relevant disability/condition tags (e.g. ['arthritis', 'wheelchair', 'stroke']) used for keyword matching |
| `price_tier` | text | — | Cost indicator: 'free', '$', '$$', '$$$' |
| `is_diy` | boolean | — | Whether this is a do-it-yourself solution the user can build/make |
| `what_made_it_work` | text | — | A real-world testimonial or personal story explaining why the |

| | | | |
|---|---|---|---|
| | | | solution was effective |
| `cover_image_url` | text | — | URL to a cover/thumbnail image for the solution card |
| `source_url` | text | — | Link to the original source (product page, video, article) |
| `is_active` | boolean | DEFAULT true | Soft-delete flag; inactive solutions are hidden from search results |
| `created_at` | timestamptz | DEFAULT now() | When the solution was added to the catalog |
| `updated_at` | timestamptz | DEFAULT now() | Auto-updated via trigger on every change |

**RLS:** Anyone can read active solutions ( `is_active = true` ).

---

## 3. solution_embeddings

**Purpose:** Stores vector embeddings (OpenAI text-embedding-ada-002, 1536 dimensions) for each solution. Powers semantic/AI-based search via pgvector cosine similarity.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `solution_id` | uuid | FK → solutions(id), UNIQUE, ON DELETE CASCADE | The solution this embedding represents (one embedding per solution) |

| | | | |
|---|---|---|---|
| `embedding` | vector(1536) | — | The 1536-dimensional vector embedding generated from the solution's text |
| `chunk_text` | text | — | The text that was embedded (typically title + description + tags concatenated) |
| `created_at` | timestamptz | DEFAULT now() | When the embedding was generated |

**RLS:** Anyone can read embeddings.

**Index:** `solution_embeddings_solution_idx` on `solution_id` for fast lookups.

---

## 4. community_ratings

**Purpose:** Stores user ratings (1-5 stars) and optional text reviews on solutions. Each user can rate a solution only once but can update their rating.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `solution_id` | uuid | FK → solutions(id), NOT NULL, ON DELETE CASCADE | The solution being rated |
| `user_id` | uuid | FK → auth.users(id), NOT NULL, ON DELETE CASCADE | The user who submitted the rating |
| `rating` | int | NOT NULL, CHECK (1-5) | Star rating from 1 (poor) to 5 (excellent) |
| `review_text` | text | — | Optional written review/comment |

| | | | |
|---|---|---|---|
| `created_at` | timestamptz | DEFAULT now() | When the rating was first submitted |
| `updated_at` | timestamptz | DEFAULT now() | Auto-updated when the rating is changed |

**Unique constraint:** `(user_id, solution_id)` — one rating per user per solution.

**RLS:** Anyone can read ratings. Users can insert, update, and delete only their own.

---

## 5. solution_likes

**Purpose:** Tracks which solutions a user has "liked" (bookmarked/favorited). Used to show a heart icon on solution cards and populate the user's liked solutions list.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `user_id` | uuid | FK → auth.users(id), NOT NULL, ON DELETE CASCADE | The user who liked the solution |
| `solution_id` | text | NOT NULL | The ID of the liked solution (stored as text) |
| `created_at` | timestamptz | DEFAULT now() | When the like was recorded |

**Unique constraint:** `(user_id, solution_id)` — a user can only like a solution once.

**RLS:** Users can view, insert, and delete only their own likes.

---

## 6. collections

**Purpose:** User-created named groups for organizing saved solutions (e.g. "Kitchen Aids", "For Mom"). Each collection has a color for visual differentiation in the UI.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `user_id` | uuid | FK → auth.users(id), NOT NULL, ON DELETE CASCADE | The user who owns this collection |
| `name` | text | NOT NULL, DEFAULT 'Collection' | Display name chosen by the user |
| `color` | text | NOT NULL, DEFAULT 'purple' | Color theme: red, orange, yellow, green, teal, blue, purple, pink |
| `created_at` | timestamptz | DEFAULT now() | When the collection was created |
| `updated_at` | timestamptz | DEFAULT now() | When the collection was last modified |

**RLS:** Users can view, insert, update, and delete only their own collections.

---

## 7. collection_items

**Purpose:** Junction table linking solutions to collections. A solution can appear in multiple collections, and a collection can contain multiple solutions.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `collection_id` | uuid | FK → collections(id) | The collection this item belongs to |
| `solution_id` | text | — | The ID of the solution in this collection |

**RLS:** Access is scoped through the parent collection — users can only manage items in collections they own.

## 8. solution_pathways

**Purpose:** Tracks a user's "solution journey" — an ordered list of solutions they plan to try, with progress tracking. Pathways can be linked to an assessment session to tie recommendations back to the original conversation.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `user_id` | uuid | FK → auth.users(id), NOT NULL, ON DELETE CASCADE | The user who owns this pathway |
| `session_id` | text | — | Optional link to the assessment session that generated these recommendations |
| `title` | text | NOT NULL | User-facing title (e.g. "Bathing Independence Plan") |
| `solutions` | jsonb | DEFAULT '[]' | Ordered array of pathway steps, each containing: `solution_id`, `step` (order number), `note` (user annotation), `completed` (boolean) |
| `status` | text | DEFAULT 'active', CHECK (active/completed/archived) | Lifecycle state of the pathway |
| `created_at` | timestamptz | DEFAULT now() | When the pathway was created |

| | | | |
|---|---|---|---|
| `updated_at` | timestamptz | DEFAULT now() | Auto-updated via trigger |

**RLS:** Users can view, insert, update, and delete only their own pathways.

---

## 9. assessment_sessions

**Purpose:** Stores the full conversation history for AI-powered needs assessments. The AI (Claude) chats with the user to understand their challenges, then extracts structured data (categories, keywords, ADL focus) used to match solutions.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Internal row ID |
| `session_id` | text | UNIQUE, NOT NULL | Client-generated session identifier used across API calls |
| `messages` | jsonb | DEFAULT '[]' | Full chat transcript as an array of `{role, content}` message objects |
| `extracted_categories` | text[] | DEFAULT '{}' | ADL categories the AI identified from the conversation (e.g. ['bathing', 'mobility']) |
| `extracted_keywords` | text[] | DEFAULT '{}' | Specific conditions/needs extracted (e.g. ['arthritis', 'wheelchair', 'balance']) |

| | | | |
|---|---|---|---|
| `adl_focus` | text | — | The primary ADL area the user needs help with |
| `role` | text | — | The user's role context: 'self' (person with disability) or 'caregiver' |
| `completed` | boolean | DEFAULT false | Whether the assessment conversation has reached a conclusion |
| `created_at` | timestamptz | DEFAULT now() | When the session started |
| `updated_at` | timestamptz | DEFAULT now() | When the session was last updated |

**RLS:** Open access (no user_id column) — sessions are identified by session_id token.

---

## 10. needs_assessments

**Purpose:** Stores the structured output of a completed assessment. While `assessment_sessions` holds the raw conversation, this table holds the final extracted profile used for solution matching.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `user_id` | uuid | FK → auth.users(id), ON DELETE SET NULL | The authenticated user (N for anonymous assessmen |

| | | | |
|---|---|---|---|
| `session_id` | text | NOT NULL | Links back to the assessment_sessions.sess |
| `role` | text | — | User's role: 'self' or 'careg |
| `categories` | text[] | DEFAULT '{}' | Finalized list of ADL categ to search |
| `keywords` | text[] | DEFAULT '{}' | Finalized list of disability/condition keywor |
| `adl_focus` | text | — | Primary ADL area for focus results |
| `severity_hints` | jsonb | DEFAULT '{}' | AI-extracted hints about severity/urgency (e.g. `{"mobility": "severe", "bathing": "moderate"}`) |
| `created_at` | timestamptz | DEFAULT now() | When the assessment was finalized |

**RLS:** Users can read their own assessments. Anonymous assessments (user_id NULL) can be inserted without auth.

---

## 11. challenge_categories

**Purpose:** Reference/lookup table for the 8 Activities of Daily Living (ADL) categories used throughout the app. Pre-seeded with data and read-only for users.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `slug` | text | UNIQUE, NOT NULL | URL-safe identifier (e.g. 'bathing', 'mobility') |
| `label` | text | NOT NULL | Human-readable name (e.g. 'Bathing', 'Mobility') |
| `description` | text | — | Explanation of what challenges this category |

| | | | |
|---|---|---|---|
| | | | covers |
| `icon_emoji` | text | — | Emoji icon for UI display |
| `sort_order` | int | DEFAULT 0 | Display ordering in the UI |

**Seeded values:** bathing, dressing, eating, mobility, toileting, transferring, communication, memory

**RLS:** Anyone can read categories. No insert/update/delete for regular users.

---

## 12. solution_feedback

**Purpose:** Records quick thumbs-up/thumbs-down feedback on solution cards. Used to measure solution quality and improve future recommendations.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `solution_id` | text | — | The solution that received feedback |
| `is_helpful` | boolean | — | `true` = thumbs up, `false` = thumbs down |
| `session_id` | text | — | Optional assessment session ID for context |
| `created_at` | timestamptz | DEFAULT now() | When the feedback was submitted |

**RLS:** Anyone can read and insert feedback (anonymous allowed).

---

## 13. diagnostic_sessions

**Purpose:** Tracks a user's journey through the structured diagnostic flow (the older, tree-based assessment before the AI chat was added). Records each step selection.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique session identifier |
| `user_id` | uuid | FK → auth.users(id) | The authenticated user (NULL for anonymous) |
| `is_authenticated` | boolean | DEFAULT false | Whether the user was logged in during this session |
| `selections_path` | jsonb | — | Ordered array of prompt IDs and selected options the user chose |
| `final_prompt_id` | text | — | The terminal prompt node where the user ended |
| `status` | text | DEFAULT 'in_progress' | Session state: 'in_progress', 'completed', 'abandoned' |
| `started_at` | timestamptz | DEFAULT now() | When the diagnostic flow began |
| `completed_at` | timestamptz | — | When the user reached a terminal node (NULL if incomplete) |

**RLS:** Users can read and update their own sessions. Anyone can insert (supports anonymous start).

---

## 14. diagnostic_prompts

**Purpose:** Defines each node in the diagnostic decision tree. Each prompt is a question shown to the user with a set of selectable options.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique prompt identifier |
| `step_order` | int | — | Numerical ordering of this step in the flow |
| `prompt_text` | text | — | The question or instruction shown to the user |
| `input_type` | text | — | UI input type: 'single_select', 'multi_select', 'text', etc. |
| `options` | jsonb | — | Array of answer options with labels and values |
| `is_root` | boolean | DEFAULT false | Whether this is the first prompt in the tree |
| `is_terminal` | boolean | DEFAULT false | Whether this prompt ends the flow and triggers results |
| `is_active` | boolean | DEFAULT true | Soft-delete flag for retired prompts |

**RLS:** Anyone can read prompts (public configuration).

---

## 15. diagnostic_paths

**Purpose:** Defines the edges in the diagnostic decision tree — which prompt leads to which next prompt based on the user's selected option.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique path identifier |

| | | | |
|---|---|---|---|
| `from_prompt_id` | uuid | FK → diagnostic_prompts(id) | The source prompt node |
| `selected_option` | text | — | The option value the user selected that triggers this path |
| `to_prompt_id` | uuid | FK → diagnostic_prompts(id) | The destination prompt node |
| `path_order` | int | — | Ordering when multiple paths exist from the same prompt |

**RLS:** Anyone can read paths (public configuration).

---

## 16. diagnostic_results

**Purpose:** Stores the matched criteria/output when a user completes a diagnostic session. Used to generate personalized solution recommendations.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `session_id` | uuid | FK → diagnostic_sessions(id) | The session that produced these results |
| `matched_criteria` | jsonb | — | JSON object containing the derived criteria (categories, keywords, severity) based on the user's path through the tree |

| | | | |
|---|---|---|---|
| `generated_at` | timestamptz | DEFAULT now() | When the results were computed |

**RLS:** Users can read results for their own sessions. Anyone can insert.

---

## 17. solution_results

**Purpose:** Caches search results returned to users — both internal (from the solutions table) and external (from web scraping). Provides a historical record of what was shown.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `session_id` | uuid | — | The session that triggered this search |
| `session_type` | text | — | Type of session: 'diagnostic' or 'assessment' |
| `source_type` | text | — | Where the result came from: 'internal', 'web', 'youtube' |
| `title` | text | — | Display title of the result |
| `url` | text | — | Link to the external resource (NULL for internal solutions) |
| `snippet` | text | — | Short preview text or description |
| `thumbnail_url` | text | — | Image thumbnail URL |
| `internal_id` | uuid | FK → solutions(id) | Link to internal solution (NULL for external results) |

| | | | |
|---|---|---|---|
| `relevance_score` | float | — | Computed relevance score (0.0 to 1.0+) from the matching algorithm |
| `fetched_at` | timestamptz | DEFAULT now() | When the result was retrieved/computed |

**RLS:** Anyone can read and insert solution results.

---

## 18. solution_tags

**Purpose:** Junction table that applies filter tags to solution results, enabling the user to filter search results by category, price, type, etc.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `result_id` | uuid | FK → solution_results(id) | The solution result being tagged |
| `filter_id` | uuid | FK → filter_options(id) | The filter tag being applied |

**RLS:** Anyone can read solution tags (public reference).

---

## 19. filter_options

**Purpose:** Reference table defining all available filter options grouped by category. Powers the filter UI on the solutions page.

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | uuid | PK, DEFAULT gen_random_uuid() | Unique identifier |
| `filter_group` | text | — | The filter category (e.g. 'price', 'source_type', 'adl_category') |

| | | | |
|---|---|---|---|
| `filter_value` | text | — | The internal value used for filtering |
| `display_label` | text | — | Human-readable label shown in the UI |
| `sort_order` | int | — | Display ordering within the filter group |
| `is_active` | boolean | DEFAULT true | Whether this filter option is currently shown |

**RLS:** Anyone can read filter options (public configuration).

---

## SQL Functions

### `match_solutions(p_categories, p_keywords, p_limit)`

**Purpose:** Keyword-based solution matching. Scores solutions by ADL category match (1.0 weight) and disability tag overlap (0.5 weight).

- **Input:** Array of category slugs, array of keyword strings, result limit (default 20)
- **Returns:** Matching solutions ordered by relevance score descending
- **Used by:** `/api/match` when no embeddings are available

### `match_solutions_semantic(query_embedding, p_categories, p_limit)`

**Purpose:** AI-powered semantic search. Combines pgvector cosine similarity (70% weight) with ADL category match (30% weight) for hybrid ranking.

- **Input:** 1536-dim vector embedding, array of category slugs, result limit (default 6)
- **Returns:** Matching solutions ordered by combined relevance score
- **Used by:** `/api/match` when OpenAI embeddings are enabled

---

## Entity Relationship Overview

```
auth.users
  ├── profiles (1:1)
  ├── solution_likes (1:N)
  ├── collections (1:N)
  │     └── collection_items (1:N) → solutions
  ├── community_ratings (1:N) → solutions
  ├── solution_pathways (1:N)
  ├── needs_assessments (1:N)
  └── diagnostic_sessions (1:N)
        └── diagnostic_results (1:N)

solutions
  ├── solution_embeddings (1:1)
  ├── community_ratings (1:N)
  └── solution_results (N:1 via internal_id)
        └── solution_tags (1:N) → filter_options

diagnostic_prompts
  └── diagnostic_paths (N:N self-referencing tree)

assessment_sessions (standalone, keyed by session_id)
```

---

*Generated for the Dayli AI project — April 2026*